

Task ASG(auto scaling group)

1. Create one VPC in N. Virginia region.
2. Create two subnets: one public subnet and one private subnet.
3. Attach an IGW to the VPC.
4. Create one public route table (RT) and one private route table.
5. Deploy a NAT gateway in the public subnet and attach the NAT gateway to the private subnet.
6. Create two instances, one in the public subnet and one in the private subnet.
7. Deploy Apache server on both EC2 instances with a sample index.html file.
8. Create one application load balancer and attach it to both EC2 instances.
9. Store application load balancer logs in S3.
10. Store the VPC flow logs in a CloudWatch log group.
11. Create monitoring dashboards to monitor CPU utilization and to monitor the Apache service.
12. If CPU utilization is more than 70%, then it should trigger auto scaling and launch new instance.

Go to **VPC** → **Subnets** → **Create subnet**

Public subnet

- VPC: my-vpc1
- Name: public-subvpc1
- AZ: for example us-east-1a
- CIDR: 10.0.1.0/24
- Create

Private subnet

- VPC: my-vpc1
- Name: private-subvpc1
- AZ: us-east-1a (same AZ as NAT for lab simplicity)

VPC dashboard

AWS Global View

Filter by VPC

vpc-0102faeb7f26a831c

my-vpc1

Owner: 207662791773

Virtual private cloud

Your VPCs

Subnets

Route tables

Subnets (2)

Find subnets by attribute or tag

VPC: vpc-0102faeb7f26a831c

Clear filters

	Name	Subnet ID	State	VPC	Block Public...	IPv4 CIDR
☐	private-subvpc1	subnet-0029457f9ff7c95e5	Available	vpc-0102faeb7f26a831c my-v...	Off	10.0.2.0/24
☐	public-subvpc1	subnet-0e9f683345750e370	Available	vpc-0102faeb7f26a831c my-v...	Off	10.0.1.0/24

Last updated

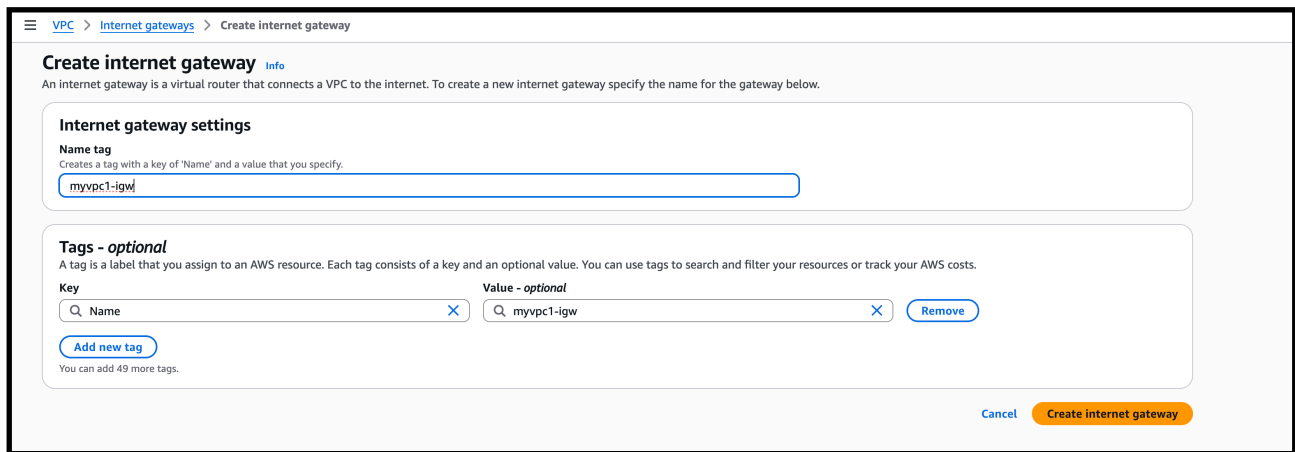
1 minute ago

Actions

3) Attach an IGW to the VPC

1. **VPC** → **Internet gateways** → **Create internet gateway**
 - Name: myvpc1-igw → Create
2. Select project-igw → **Actions** → **Attach to VPC** → choose project-vpc → Attach.

This IGW gives internet to resources in **public** subnet (through route table).



Create internet gateway [Info](#)

An internet gateway is a virtual router that connects a VPC to the internet. To create a new internet gateway specify the name for the gateway below.

Internet gateway settings

Name tag
Creates a tag with a key of 'Name' and a value that you specify.

myvpc1-igw

Tags - optional
A tag is a label that you assign to an AWS resource. Each tag consists of a key and an optional value. You can use tags to search and filter your resources or track your AWS costs.

Key **Value - optional**

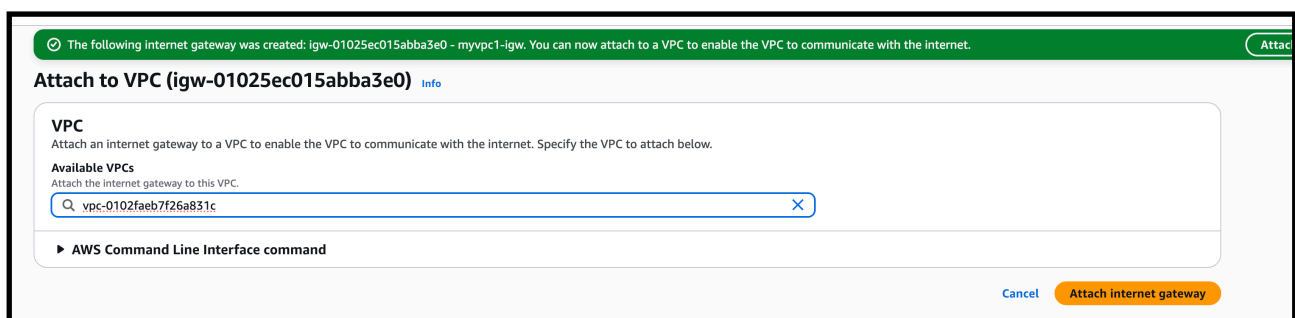
Q Name X Q myvpc1-igw X Remove

Add new tag

You can add 49 more tags.

Cancel Create internet gateway

—> attached the igw to the vpc



The following internet gateway was created: igw-01025ec015abba3e0 - myvpc1-igw. You can now attach to a VPC to enable the VPC to communicate with the internet. [Attach](#)

Attach to VPC (igw-01025ec015abba3e0) [Info](#)

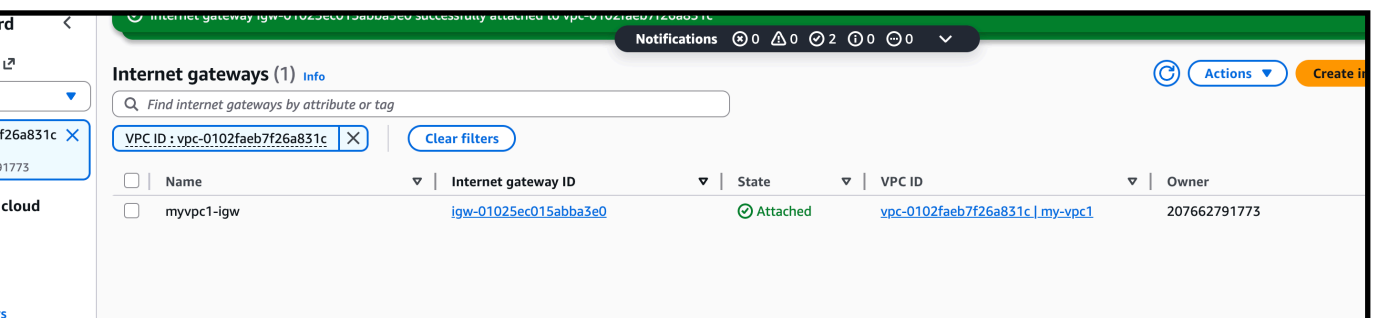
VPC
Attach an internet gateway to a VPC to enable the VPC to communicate with the internet. Specify the VPC to attach below.

Available VPCs
Attach the internet gateway to this VPC.

Q vpc-0102faeb7f26a831c X

► AWS Command Line Interface command

Cancel Attach internet gateway



Internet gateways (1) [Info](#)

Find internet gateways by attribute or tag

VPC ID: vpc-0102faeb7f26a831c X Clear filters

☐	Name	Internet gateway ID	State	VPC ID	Owner
☐	myvpc1-igw	igw-01025ec015abba3e0	Attached	vpc-0102faeb7f26a831c my-vpc1	207662791773

4) Create one public route table (RT) and one private route table.

VPC → Route tables → Create route table

- Name: `public-rt`
- VPC: `my-vpc1`
- Create

Select `public-rt` → Routes tab → Edit routes:

- Add route:
 - Destination: `0.0.0.0/0`
 - Target: Internet Gateway (`project-igw`)
- Save

Subnet associations tab → Edit:

- Select `public-subnet-1`
- Save

The screenshot displays the AWS Management Console interface for configuring a route table. On the left, a sidebar lists navigation options under 'Virtual private cloud', including 'Route tables'. The main content area shows a list of route tables, with one selected: 'rtb-0f06a199121e4017f'. Below this, the 'Routes' tab is active, showing a table with two routes. The first route has a destination of '0.0.0.0/0' and a target of 'igw-01025ec015abba3e0', both with an 'Active' status. The second route has a destination of '10.0.0.0/16' and a target of 'local', also with an 'Active' status. The interface includes search bars, filters, and pagination controls.

Destination	Target	Status	Propagated	Route Origin
0.0.0.0/0	igw-01025ec015abba3e0	Active	No	Create Route
10.0.0.0/16	local	Active	No	Create Route Table

—> Private Route Table

1. Create route table again:

- Name: myprivate-route
- VPC: my-vpc1 → Create

2. (Routing to NAT we'll do after creating NAT)

3. Subnet associations:

- Associate private-subnet-1 to private-rt

Create route table [Info](#)

A route table specifies how packets are forwarded between the subnets within your VPC, the internet, and your VPN connection.

Route table settings

Name - optional

Create a tag with a key of 'Name' and a value that you specify.

VPC

The VPC to use for this route table.

Tags

A tag is a label that you assign to an AWS resource. Each tag consists of a key and an optional value. You can use tags to search and filter your resources or track your AWS costs.

Key

Value - optional

[Remove](#)[Add new tag](#)

You can add 49 more tags.

[Cancel](#)[Create route table](#)

—> private route table is created

The screenshot shows the AWS Management Console interface. On the left, the 'Virtual private cloud' sidebar is visible, with 'Route tables' selected. The main content area displays the 'Create route table' page, which has been completed. The 'Route table settings' section shows the name 'myprivate-route' and the VPC 'vpc-0102faeb7f26a831c (my-vpc1)'. The 'Tags' section shows a tag with key 'Name' and value 'myprivate-route'. The 'Routes' section shows a single route with destination '10.0.0.0/16' and target 'local', with status 'Active'. The 'Subnet associations' section shows the route table is associated with 'private-subnet-1'.

Name	Route table ID	Explicit subnet associ...	Edge associations	Main	VPC	Owner ID
mypublic-route	rtb-0f06a199121e4017f	-	-	Yes	vpc-0102faeb7f26a831c my-v...	207662791773
myprivate-route	rtb-04d22a680c60f4c2d	-	-	No	vpc-0102faeb7f26a831c my-v...	207662791773

rtb-04d22a680c60f4c2d / myprivate-route

Destination	Target	Status	Propagated	Route Origin
10.0.0.0/16	local	Active	No	Create Route Table

5) Deploy a NAT gateway in the public subnet and attach the NAT gateway to the private subnet.

Create NAT Gateway in public subnet

1. Go to **VPC** → **NAT gateways** → **Create NAT gateway**
 - Name: `my-nat-gateway`
 - Vpc
 - Elastic IP: click **Allocate Elastic IP** and choose it
 - Create
2. After NAT is **Available**, edit **private-rt** routes:
 - Select **private-rt** → **Routes** → **Edit routes** → **Add route**
 - Destination: `0.0.0.0/0`
 - Target: **NAT Gateway (project-nat)**
 - Save

Now **private subnet** can go out to internet (yum/apt updates) but internet cannot directly reach it.

Elastic IP address 98.86.73.37 (eipalloc-063c1bc086842fc0c) allocated.

Create NAT gateway [Info](#)

A highly available, managed Network Address Translation (NAT) service that instances in private subnets can use to connect to services in other VPCs, on-premises networks, or the internet.

NAT gateway settings

Name - optional
Create a tag with a key of 'Name' and a value that you specify.

The name can be up to 256 characters long.

Availability mode [Info](#)
Choose whether to deploy across all zones in the region or restrict to a single availability zone.

☐ **Regional - new**
Scales automatically across all regional AZs, simplifying management for multi AZ deployments.

☒ **Zonal**
Provides granular control within a specific availability zone, adhering to subnet level settings.

Subnet
Select a subnet in which to create the NAT gateway.

Connectivity type
Select a connectivity type for the NAT gateway.

☒ **Public**

☐ Private

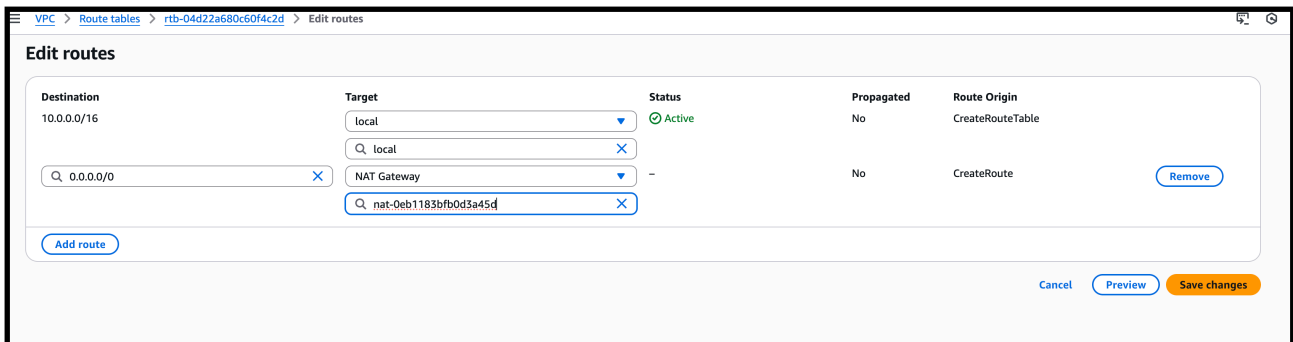
Elastic IP allocation ID [Info](#)
Assign an Elastic IP address to the NAT gateway.

[Allocate Elastic IP](#)

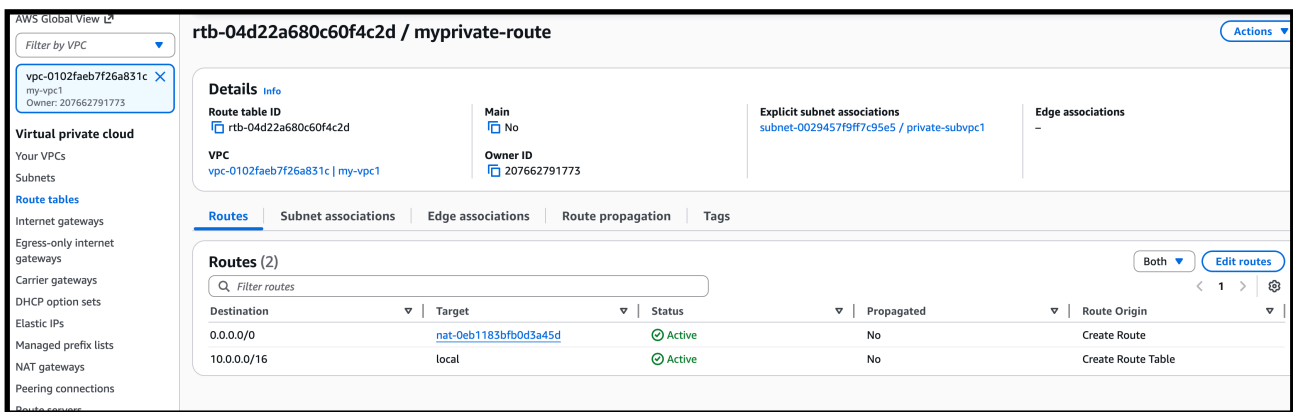
Additional settings [Info](#)

Tags

—> go to private route and attach the nat gateway



—> nat gate way is attached to the private route table



6) Create two instances, one in the public subnet and one in the private subnet.

mypublic-ec2	i-0d93680da9d8c44b	terminated	t3.micro	3/3 checks passed	us-east-1b
private-ec2	i-0781309fa29f7f3ba	Running	t3.micro	3/3 checks passed	us-east-1b
public-ec2	i-00205791b692d4418	Running	t3.micro	3/3 checks passed	us-east-1b

Steps:

—> Create EC2 instances (1 public, 1 private)

—> select the vpc which we created and for public ec2 select public subnet

—> and for same in private instance choose created vpc and select the private subnet of the vpc

—> and launch the instance

—> before launching I have added the script for installing httpd apache in data centre from task 7

7)Deploy Apache server on both EC2 instances with a sample index.html file.

EC2 > Instances > Launch an instance

Metadata version | [Info](#)

V2 only (token required) ▼

⚠ For V2 requests, you must include a session token in all instance metadata requests. Applications or agents that use V1 for instance metadata access will break.

Metadata response hop limit | [Info](#)

2

Allow tags in metadata | [Info](#)

Select ▼

User data - *optional* | [Info](#)

Upload a file with your user data or enter it in the field.

⬆ Choose file

```
#!/bin/bash
yum -y update
yum -y install httpd
systemctl enable httpd
systemctl start httpd
echo "<h1>Hello from PUBLIC EC2 - $(hostname)</h1>" > /var/www/html/index.html
|
```

—> for public ec2 added the above script

—> for private ec2 added the below data

Metadata response hop limit

Info

2

Allow tags in metadata

Info

Select

User data - optional

Info

Upload a file with your user data or enter it in the field.

Choose file

```
#!/bin/bash
yum -y update
yum -y install httpd
systemctl enable httpd
systemctl start httpd
echo "<h1>Hello from PRIVATE EC2 - $(hostname)</h1>" > /var/www/html/index.html
```

—> now to check the httpd index.html page is running login to the public ec2

```
waseemakram@waseem downloads % ssh -i "virginia.pem" ec2-user@44.210.124.122
```

```
#_
~\ ##### Amazon Linux 2023
~~ \#####\
~~ \###|
~~ \#/ _____
~~      V~' '->
~~~~
    ~._. _/_
        /_/_
          m/'
```

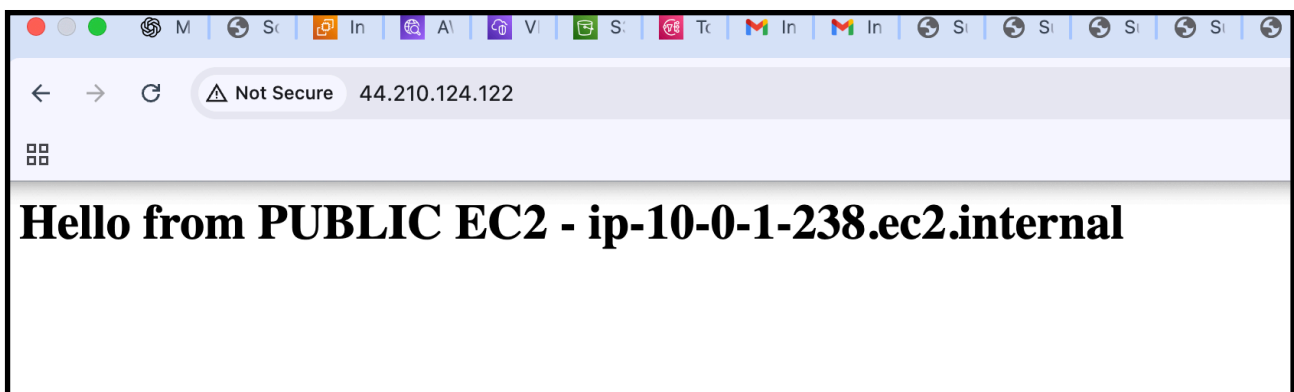
Last login: Tue Dec 9 11:13:09 2025 from 14.192.14.60

—-> we can see through image the apache is running

```
Warning: Permanently added '44.210.124.122' (ED25519) to the list of known hosts.
#_
~\ ##### Amazon Linux 2023
~~ \#####\
~~ \###|
~~ \#/ ____ https://aws.amazon.com/linux/amazon-linux-2023
~~ V~' '->
~~~
~~~.._/_/_/
~~~/_/_/_/
~~~/_m/'

[[ec2-user@ip-10-0-1-238 ~]$ systemctl status httpd
● httpd.service - The Apache HTTP Server
   Loaded: loaded (/usr/lib/systemd/system/httpd.service; enabled; preset: disabled)
   Active: active (running) since Tue 2025-12-09 11:12:13 UTC; 1min 9s ago
     Docs: man:httpd.service(8)
  Main PID: 3194 (httpd)
    Status: "Total requests: 0; Idle/Busy workers 100/0; Requests/sec: 0; Bytes served/sec: 0 B/sec"
     Tasks: 177 (limit: 1067)
    Memory: 13.3M
       CPU: 123ms
    CGroup: /system.slice/httpd.service
            └─3194 /usr/sbin/httpd -DFOREGROUND
            └─3288 /usr/sbin/httpd -DFOREGROUND
            └─3294 /usr/sbin/httpd -DFOREGROUND
            └─3295 /usr/sbin/httpd -DFOREGROUND
```

—-> now browse the public ip of the public ec2 to check index.html sample page



- > now lets login to our private ec2
- > step to login in private ec2
- > first we have to copy the pem key by cat and
- > create a file name test.pem in public instance
- > and add the pem key content inside the test.pem file
- > give chmod 400 for permission

```
waseemakram@waseem downloads % ssh -i "virginia.pem" ec2-user@44.210.124.122
#_
~\_ #####_      Amazon Linux 2023
~~ \#####\
~~  \###|
~~   \#/  ____  https://aws.amazon.com/linux/amazon-linux-2023
~~    V~' '->
~~~
~~~_._. _/
~~  _/  _/
~~   _/m/'

Last login: Tue Dec  9 11:13:09 2025 from 14.192.14.60
[ec2-user@ip-10-0-1-238 ~]$ vi test.pem
[ec2-user@ip-10-0-1-238 ~]$ chmod 400 test.pem
[ec2-user@ip-10-0-1-238 ~]$ ssh -i test.pem ec2-user@10.0.2.201
The authenticity of host '10.0.2.201 (10.0.2.201)' can't be established.
ED25519 key fingerprint is SHA256:XQfvJG/6wj7/SAYranKYbnt3dF3t2bsELtztCTI9qsA.
This key is not known by any other names
Are you sure you want to continue connecting (yes/no/[fingerprint])? yes
Warning: Permanently added '10.0.2.201' (ED25519) to the list of known hosts.
#_
~\_ #####_      Amazon Linux 2023
~~ \#####\
~~  \###|
~~   \#/  ____  https://aws.amazon.com/linux/amazon-linux-2023
~~    V~' '->
~~~
~~~_._. _/
~~  _/  _/
~~   _/m/'
```

- > and ssh -i “test.pem” ec2-user@<private ip of private route>
- > by this we can successfully login to our private ec2
- > now to check if it has httpd as we have connected nat gate way I should have internet access

```
[ec2-user@ip-10-0-2-201 ~]$ systemctl status httpd
o httpd.service - The Apache HTTP Server
   Loaded: loaded (/usr/lib/systemd/system/httpd.service; disabled; preset: disabled)
   Active: inactive (dead)
     Docs: man:httpd.service(8)
```

- > httpd is available in side our private ec2

—> After that click next and you will be in register target step and we have to add the instances in register target

Protocol for communication between the load balancer and targets.
HTTP

Port number where targets receive traffic. Can be overridden for individual targets during registration.
80
1-65535

IP address type
Only targets with the indicated IP address type can be registered to this target group.

☒ IPv4
Each instance has a default network interface (eth0) that is assigned the primary private IPv4 address. The instance's primary private IPv4 address is the one that will be applied to the target.

☐ IPv6
Each instance you register must have an assigned primary IPv6 address. This is configured on the instance's default network interface (eth0). [Learn more](#)

VPC
Select the VPC with the instances that you want to include in the target group. Only VPCs that support the IP address type selected above are available in this list.
vpc-0102faeb7f26a831c (my-vpc1)
10.0.0.0/16 [Create VPC](#)

Protocol version
☒ HTTP1
Send requests to targets using HTTP/1.1. Supported when the request protocol is HTTP/1.1 or HTTP/2.

☐ HTTP2
Send requests to targets using HTTP/2. Supported when the request protocol is HTTP/2 or gRPC, but gRPC-specific features are not available.

☐ gRPC
Send requests to targets using gRPC. Supported when the request protocol is gRPC.

Health checks
The associated load balancer periodically sends requests, per the settings below, to the registered targets to test their status.

Health check protocol
HTTP

Register both instances:

- public-ec2
- private-ec2

Click **Create target group**

EC2 > Target groups > Create target group

Step 1
● Create target group

Step 2 - recommended
● **Register targets**

Step 3
○ Review and create

Register targets - recommended
This is an optional step to create a target group. However, to ensure that your load balancer routes traffic to this target group you must register your targets.

Available instances (2/2)

Filter instances

☒	Instance ID	Name	State	Security groups	Zone	Private IPv4 ac
☒	i-0781309fa297f3ba	private-ec2	Running	launch-wizard-8	us-east-1b	10.0.2.201
☒	i-00205791b692d4418	public-ec2	Running	launch-wizard-7	us-east-1b	10.0.1.238

2 selected

Ports for the selected instances
Ports for routing traffic to the selected instances.
80
1-65535 (separate multiple ports with commas)

[Include as pending below](#)

EC2 > Target groups > Create target group

Register targets

Step 3

Review and create

Available instances (2/2)

Filter instances

< 1 > ⚙

☒	Instance ID	Name	State	Security groups	Zone	Private IPv4 address
☒	i-0781309fa29f7f3ba	private-ec2	Running	launch-wizard-8	us-east-1b	10.0.2.201
☒	i-00205791b692d4418	public-ec2	Running	launch-wizard-7	us-east-1b	10.0.1.238

2 selected

Ports for the selected instances

Ports for routing traffic to the selected instances.

1-65535 (separate multiple ports with commas)

Include as pending below

2 selections are now pending below. Include more or register targets when ready.

Review targets

Targets (2)

Filter targets

Show only pending

Remove all pending

< 1 > ⚙

Instance ID	Name	Port	State	Security groups	Zone	Private IPv4 address	Subnet ID	Launch time
i-0781309fa29f7f3ba	private-ec2	80	Running	launch-wizard-8	us-east-1b	10.0.2.201	subnet-0029457f9f7c95e5	December 9, 2025, 1
i-00205791b692d4418	public-ec2	80	Running	launch-wizard-7	us-east-1b	10.0.1.238	subnet-0e9f683345750e370	December 9, 2025, 1

—-> after clicking next we will on review Stage of the target groups
after review we have to simply click create target group

EC2 > Target groups > Create target group

Step 1

Create target group

Step 2 - recommended

Register targets

Step 3

Review and create

Review and create

Review your target group configuration before creating

Step 1: Target group details

Edit

Target group details

Name

tg-web

VPC

vpc-0102faeb7f26a831c

Target type

Instance

IP address type

IPv4

Protocol : Port

HTTP: 80

Protocol version

HTTP1

Health check details

Health check protocol

HTTP

Timeout

5 seconds

Health check path

/

Healthy threshold

5

Health check port

traffic-port

Unhealthy threshold

2

Interval

30 seconds

Success codes

200

Step 2: Register targets

Edit

Targets (2)

Instance ID	Name	Port	Zone
i-0781309fa29f7f3ba	private-ec2	80	us-east-1b
i-00205791b692d4418	public-ec2	80	us-east-1b

—> so currently we have created a “tg-web” named target groups

The screenshot displays the AWS Management Console for the 'tg-web' target group. The console shows the target group is in the 'Unused' state with 2 targets. The details section shows the target type is 'Instance', protocol is 'HTTP', and port is '80'. The VPC is 'vpc-0102faeb7f26a831c'. The distribution of targets by Availability Zone (AZ) shows 2 targets in us-east-1b (us-east-1b-us-east-1a) and 0 targets in us-east-1c (us-east-1a). The registered targets table shows 2 matches, with the first target being 'public-ec2' in the 'us-east-1b (us-east-1a)' zone, with a health status of 'Unused'.

Now we have to create load balancer

—->

Step 2: Create ALB

1. Go to **EC2** → **Load Balancers**
2. Click **Create Load Balancer** → **Application Load Balancer**
3. Name:
my-alb
- 4.
5. Scheme: **Internet-facing**
6. IP type: **IPv4**
7. Select **2 public subnets** (must have IGW route)
8. Security group:
 - Allow **HTTP (80)** from anywhere

9. Listener:
 - HTTP:80 → select **my-target-group**
10. Click **Create Load Balancer**

Create Application Load Balancer [Info](#)

The Application Load Balancer distributes incoming HTTP and HTTPS traffic across multiple targets such as Amazon EC2 instances, microservices, and containers, based on request attributes. When evaluates the listener rules in priority order to determine which rule to apply, and if applicable, it selects a target from the target group for the rule action.

► How Application Load Balancers work

Basic configuration

Load balancer name
Name must be unique within your AWS account and can't be changed after the load balancer is created.

my-alb

A maximum of 32 alphanumeric characters including hyphens are allowed, but the name must not begin or end with a hyphen.

Scheme | [Info](#)
Scheme can't be changed after the load balancer is created.

☒ **Internet-facing**

- Serves internet-facing traffic.
- Has public IP addresses.
- DNS name resolves to public IPs.
- Requires a public subnet.

☐ **Internal**

- Serves internal traffic.
- Has private IP addresses.
- DNS name resolves to private IPs.
- Compatible with the **IPv4** and **Dualstack** IP address types.

Load balancer IP address type | [Info](#)
Select the front-end IP address type to assign to the load balancer. The VPC and subnets mapped to this load balancer must include the selected IP address types. Public IPv4 addresses have an additional cost.

☒ **IPv4**

Includes only IPv4 addresses.

☐ **Dualstack**

Includes IPv4 and IPv6 addresses.

☐ **Dualstack without public IPv4**

Includes a public IPv6 address, and private IPv4 and IPv6 addresses. Compatible with **internet-facing** load balancers only.

—> select the vpc which we created and add its availability zones
Subnets

Network mapping [Info](#)

The load balancer routes traffic to targets in the selected subnets, and in accordance with your IP address settings.

VPC | [Info](#)
The load balancer will exist and scale within the selected VPC. The selected VPC is also where the load balancer targets must be hosted unless routing to Lambda or on-premises targets, or if using VPC peering. To confirm the VPC for your targets, view [target groups](#).

vpc-0102faeb7f26a831c (my-vpc1)
10.0.0.0/16

 [Create VPC](#)

IP pools | [Info](#)
You can optionally choose to configure an IPAM pool as the preferred source for your load balancers IP addresses. Create or view Pools in the [Amazon VPC IP Address Manager console](#).

☐ **Use IPAM pool for public IPv4 addresses**
The IPAM pool you choose will be the preferred source of public IPv4 addresses. If the pool is depleted IPv4 addresses will be assigned by AWS.

Availability Zones and subnets | [Info](#)
Select at least two Availability Zones and a subnet for each zone. A load balancer node will be placed in each selected zone and will automatically scale in response to traffic. The load balancer routes traffic to targets in the selected Availability Zones only.

☒ **us-east-1b (use1-az2)**

Subnet
Only CIDR blocks corresponding to the load balancer IP address type are used. At least 8 available IP addresses are required for your load balancer to scale efficiently.

subnet-0e9f683345750e370
IPv4 subnet CIDR: 10.0.1.0/24

public-subvpc1

☒ **us-east-1d (use1-az6)**

Subnet
Only CIDR blocks corresponding to the load balancer IP address type are used. At least 8 available IP addresses are required for your load balancer to scale efficiently.

subnet-07e9372def974128c
IPv4 subnet CIDR: 10.0.3.0/24

mypublic2-vpc1

—> now attach the target group which we created
“Tg-web”

Listeners and routing [Info](#)

A listener is a process that checks for connection requests using the port and protocol you configure. The rules that you define for a listener determine how the load balancer routes requests to its registered targets.

▼ Listener HTTP:80

Protocol Port
1-65535

Default action [Info](#)

The default action is used if no other rules apply. Choose the default action for traffic on this listener.

Routing action

☒ Forward to target groups ☐ Redirect to URL ☐ Return fixed response

Forward to target group [Info](#)

Choose a target group and specify routing weight or [create target group](#) [↗](#).

Target group HTTP ☒ **Weight** **Percent**
Target type: Instance, IPv4 | Target stickiness: Off 0-999

[+ Add target group](#)

You can add up to 4 more target groups.

Target group stickiness [Info](#)

Enables the load balancer to bind a user's session to a specific target group. To use stickiness the client must support cookies. If you want to bind a user's session to a specific target, turn on the Target Group attribute Stickiness.

☐ Turn on target group stickiness

Listener tags - optional

Consider adding tags to your listener. Tags enable you to categorize your AWS resources so you can more easily manage them.

—> and next step click create

▼ Instances Instances Instance Types Launch Templates Spot Requests Savings Plans Reserved Instances Dedicated Hosts Capacity Reservations Capacity Manager New	Load balancers (1) What's new? Refresh Actions Create load balancer									
	Elastic Load Balancing scales your load balancer capacity automatically in response to changes in incoming traffic.									
	Filter load balancers									
	☐	Name	State	Type	Scheme	IP address type	VPC ID ↗	Availability Zones		
▼ Images AMIs AMI Catalog	☐	my-alb	Active	application	Internet-facing	IPv4	vpc-0102faeb7f26a831c	2 Availability Zones		
▼ Elastic Block Store										

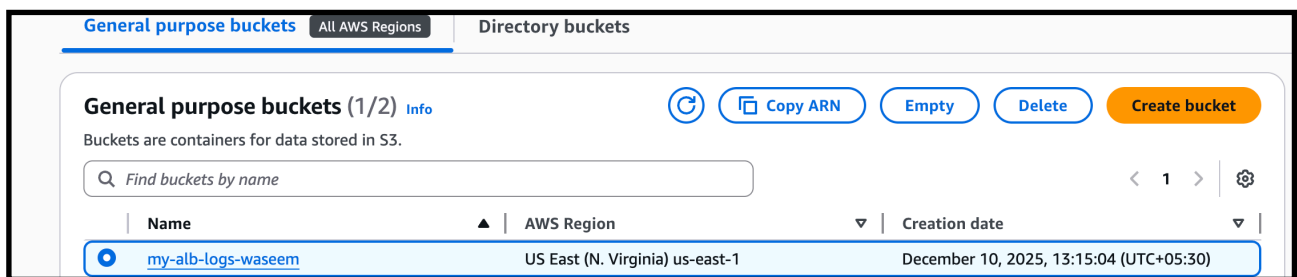
By this we have successfully created a application load balancer for
and we have attached it with subnets (ec2)

9) Store application load balancer logs in S3.

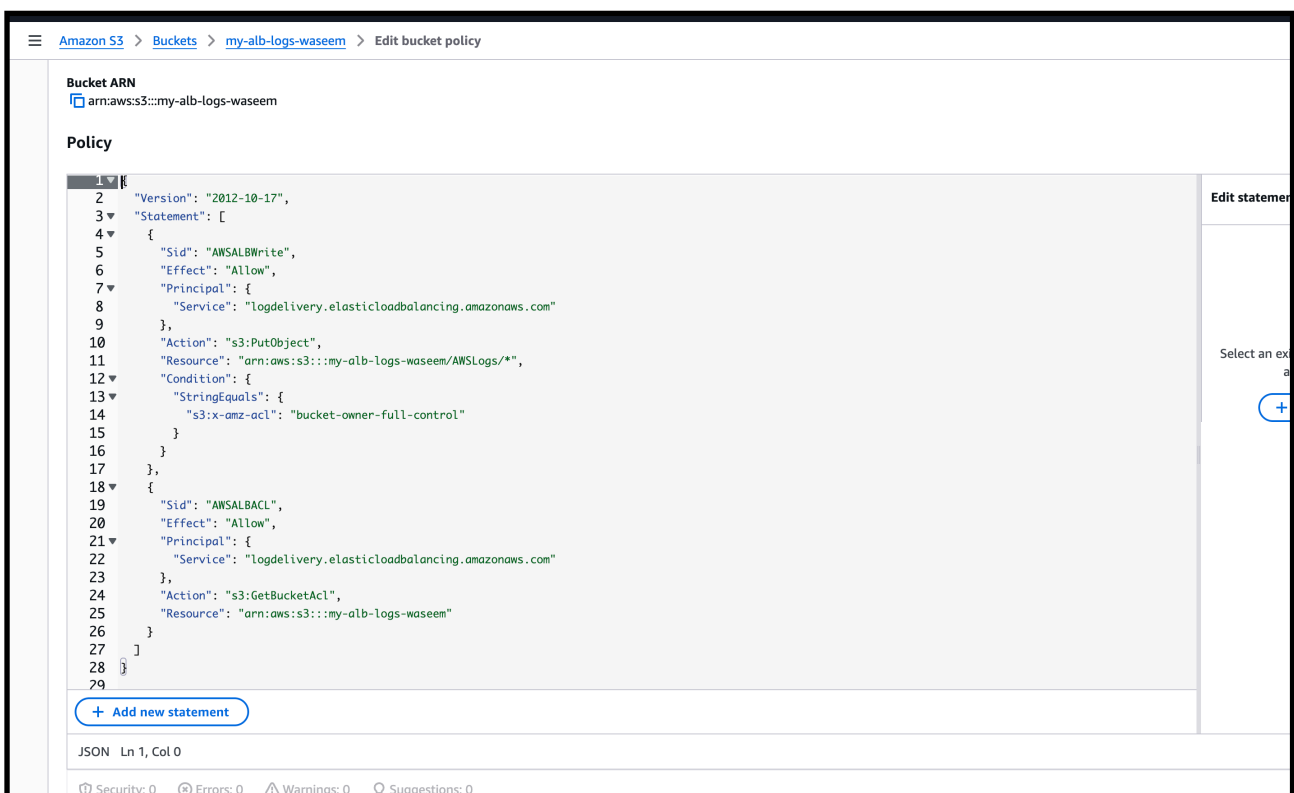
—> to store the logs of load balancer we have to create a s3 bucket Named with that load balancer

—> I given the “my-alb-logs-waseem” as bucket name

1. Block public access → Keep ON (default)
2. Create bucket



—> give the bucket policy for storing logs from load balancer



—-> now go to the load balancer —> my-alb—> and look for attributes

Enable ALB Logs

1. Go to **EC2** → **Load balancers** → **my-alb**
2. Click **Attributes**
3. Enable:
 - **Access Logs** → **ON**
4. Provide S3 bucket name: browse the name of the bucket which we created

EC2 > Load balancers > my-alb > Edit load balancer attributes

☐ Client port preservation
Indicates whether the X-Forwarded-For header should preserve the source port that the client used to connect to the load balancer.

☐ Preserve host header
Indicates whether the Application Load Balancer should preserve the Host header in the HTTP request and send it to targets without any change.

Availability Zone routing configuration

☒ Cross-zone load balancing
Cross-zone load balancing is always on for Application Load Balancers. However, you can turn it off for a specific target group using target group attributes.

ARC zonal shift integration | [Info](#)
Controls whether Amazon Application Recovery Controller (ARC) zonal shift is available to the load balancer.

☒ Disable - Default
Zonal shift will not be available to the load balancer.

☐ Enable
Zonal shift will be available to the load balancer.

Protection

☐ Deletion protection
To prevent your load balancer from being deleted accidentally, turn on deletion protection. If you turn on deletion protection, you must turn it off before you can delete the load balancer.

Monitoring

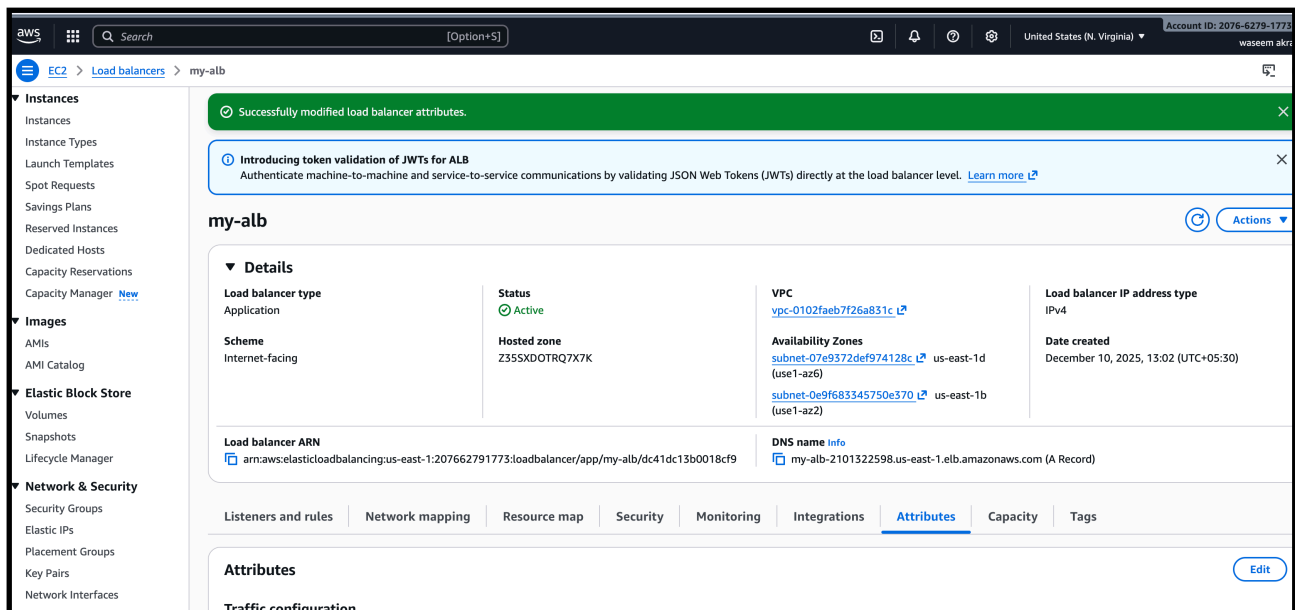
☒ Access logs
Access logs deliver detailed logs of all requests made to your Elastic Load Balancer. Choose an existing S3 location. If you don't specify a prefix, the logs are stored in the root of the bucket.

S3 URI

Format: s3://bucket/prefix

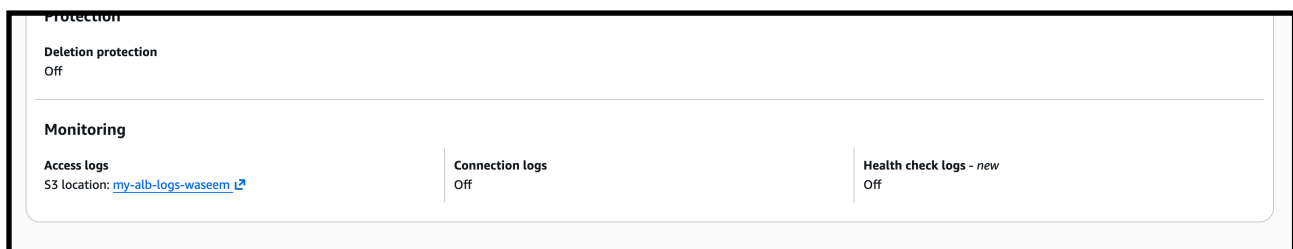
☐ Connection logs
Connection logs deliver detailed logs of all connections made to your Elastic Load Balancer. Choose an existing S3 location. If you don't specify a prefix, the logs are stored in the root of the bucket.

—-> after this step click next

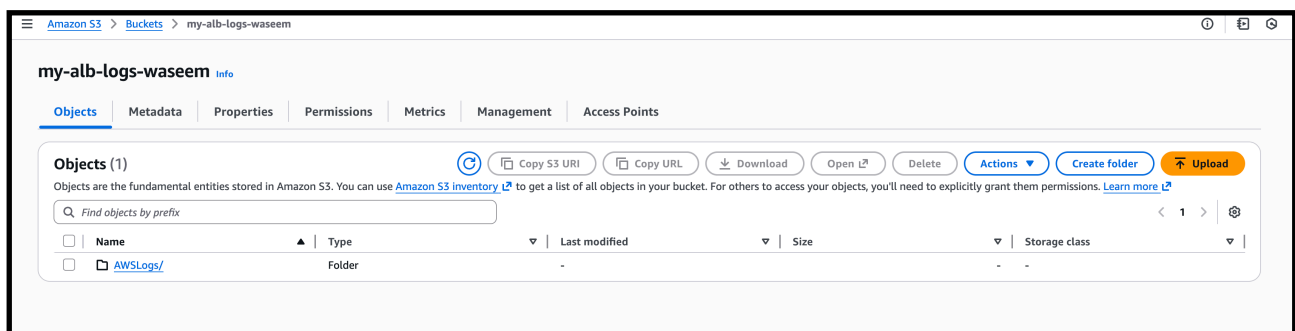


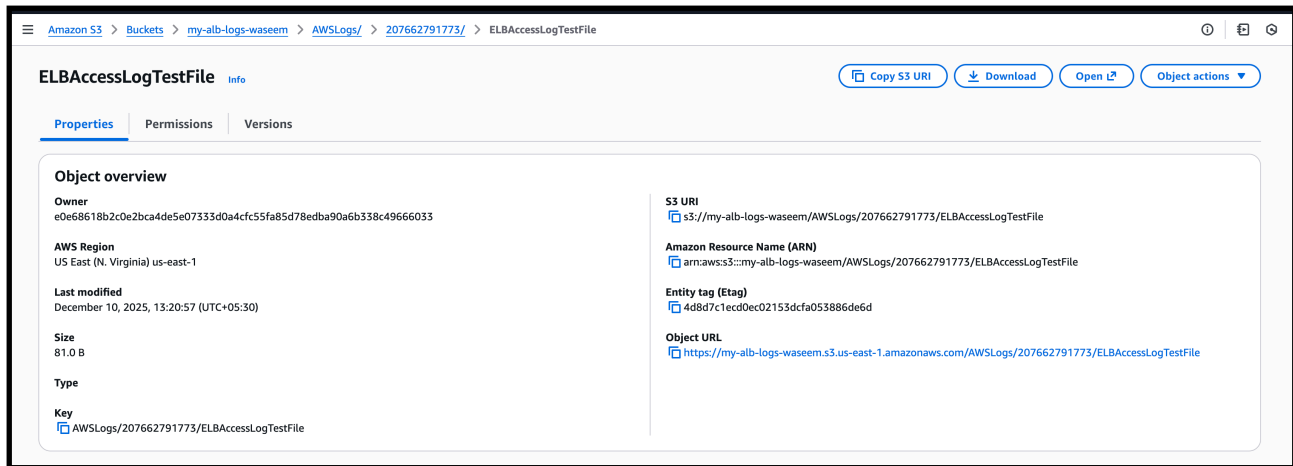
Done.

—> After few minutes logs will appear inside S3.



—> to view this logs go to the bucket and view object





—-> open all logs till end

—-> by this our load balancer logs are being stored in our s3 bucket

10) Store the VPC flow logs in a CloudWatch log group.

It will record all incoming + outgoing traffic inside your VPC
(like who connected, from which IP, which port, etc.)

— steps:~

- ✓ Create VPC Flow Logs
- ✓ Send flow logs into **CloudWatch Log Group**
- ✓ Not to S3 bucket

1: Open VPC Console

Go to:

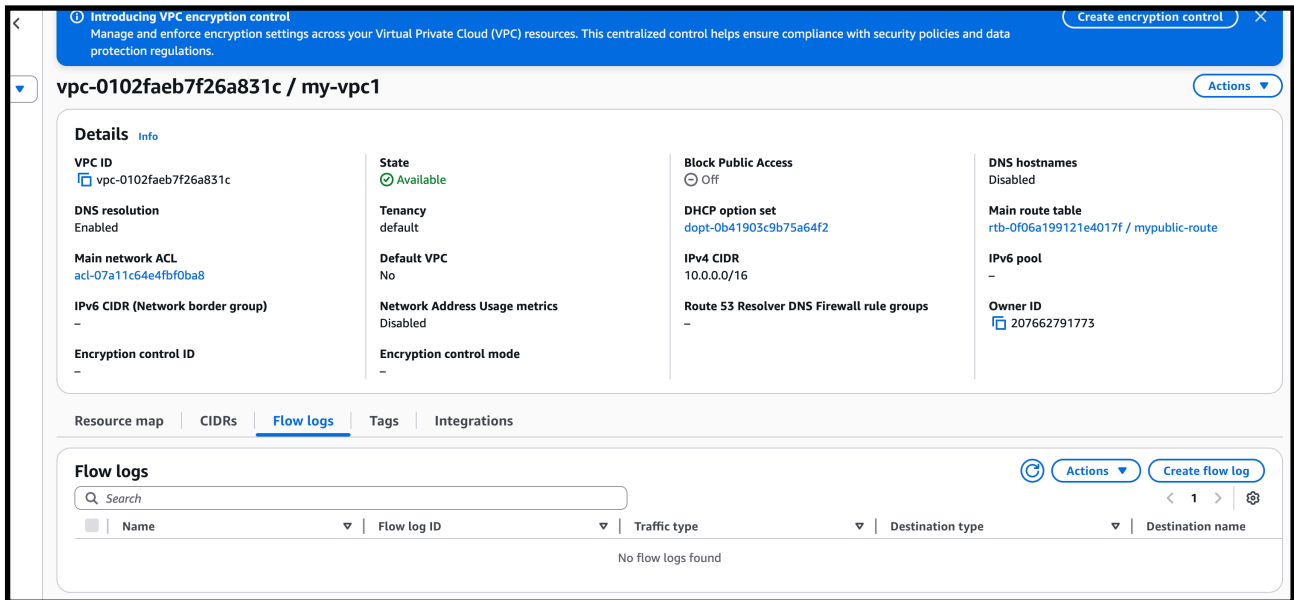
AWS → VPC → Your VPCs

Select **your VPC**

2: Create Flow Log

After selecting your VPC:

Left side → **Flow logs** → Click **Create flow log**



—>Correct Settings You Should Select

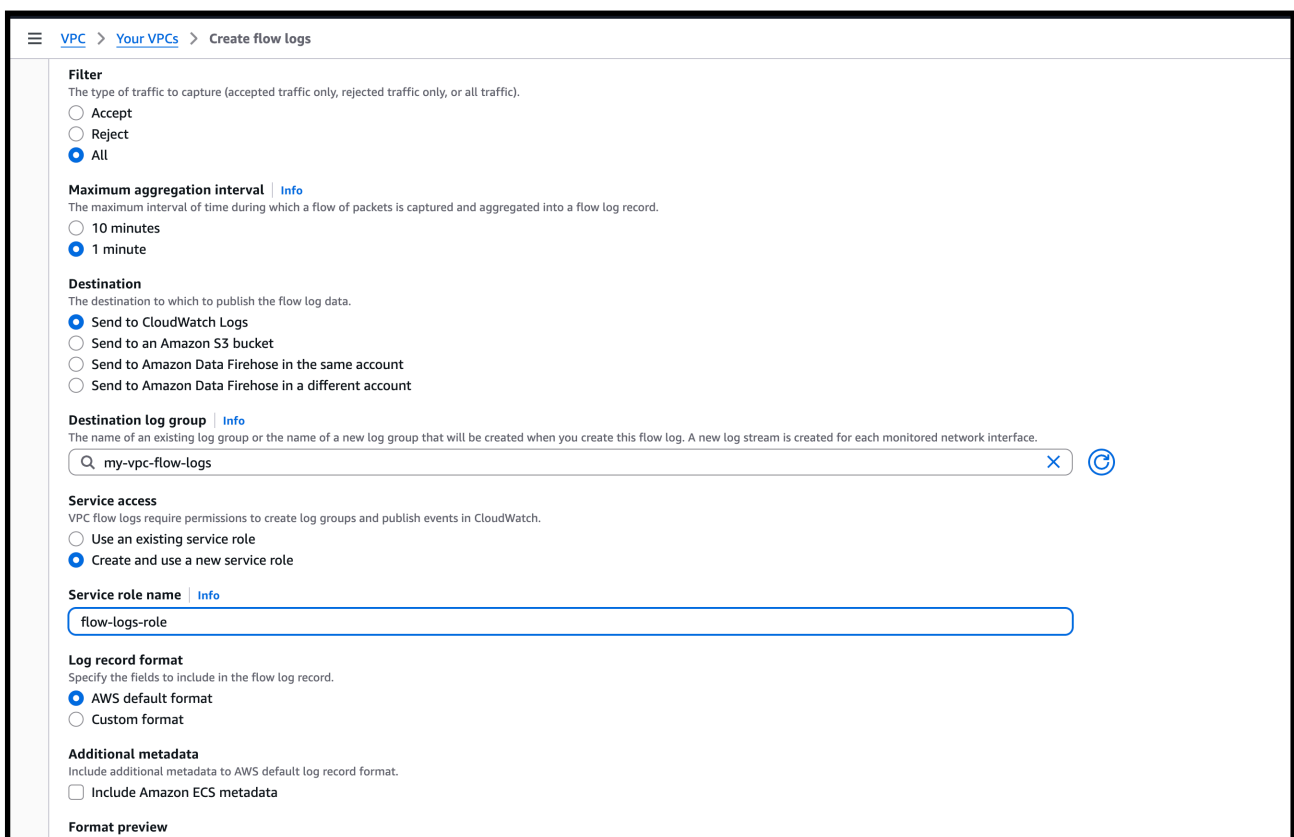
✓ **Filter: All**

✓ **Max aggregation:1 minute**

✓ **Destination :Send to CloudWatch Logs**

✓ **Destination log group : my-vpc-flow-logs**

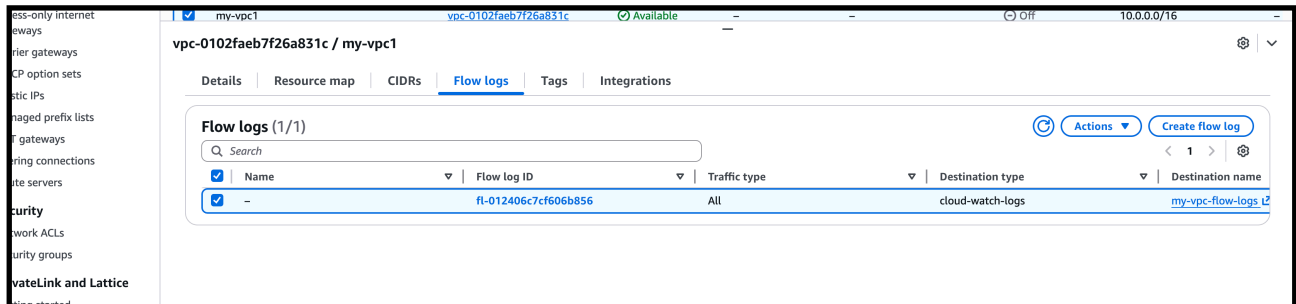
==> give a role name and keep other settings default



Step 5: Click Create

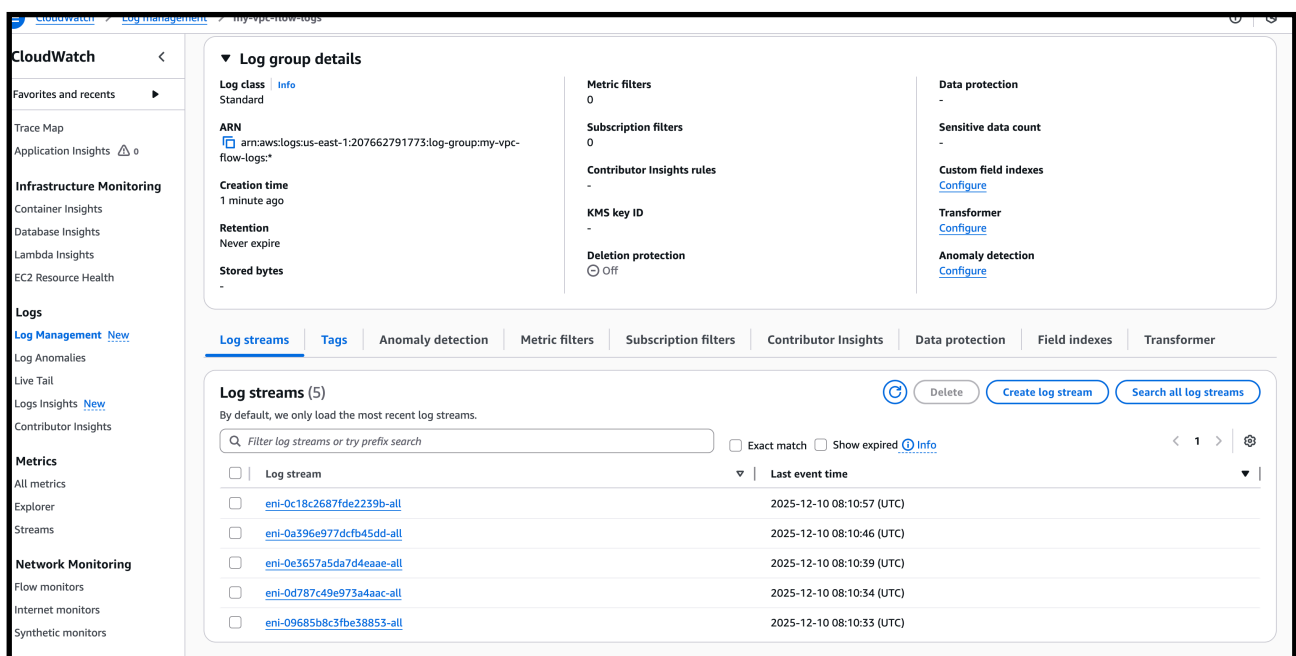
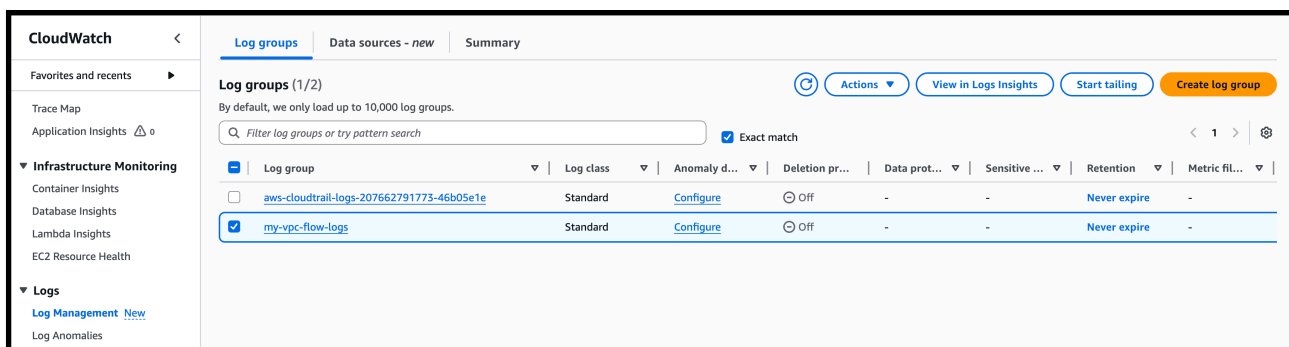
That's it.

Your VPC Flow Logs will start coming inside CloudWatch log group.



—> What You Should See Next
Inside CloudWatch:

Log Groups → /my-vpc-flow-logs



—> VPC Flow Logs are **WORKING** properly and being sent to **CloudWatch Log Group /my-vpc-flow-logs**.

11) Create monitoring dashboards to monitor CPU utilization and to monitor the Apache service.

must create a dashboard with:

(A) CPU Utilization Monitoring for both EC2 Instances

(B) Apache HTTPD Service Monitoring

Let's start step-by-step.

—> **Go to CloudWatch → Dashboards**

Left side → **Dashboards → Create dashboard**

Name it:

my-monitor-dashboard

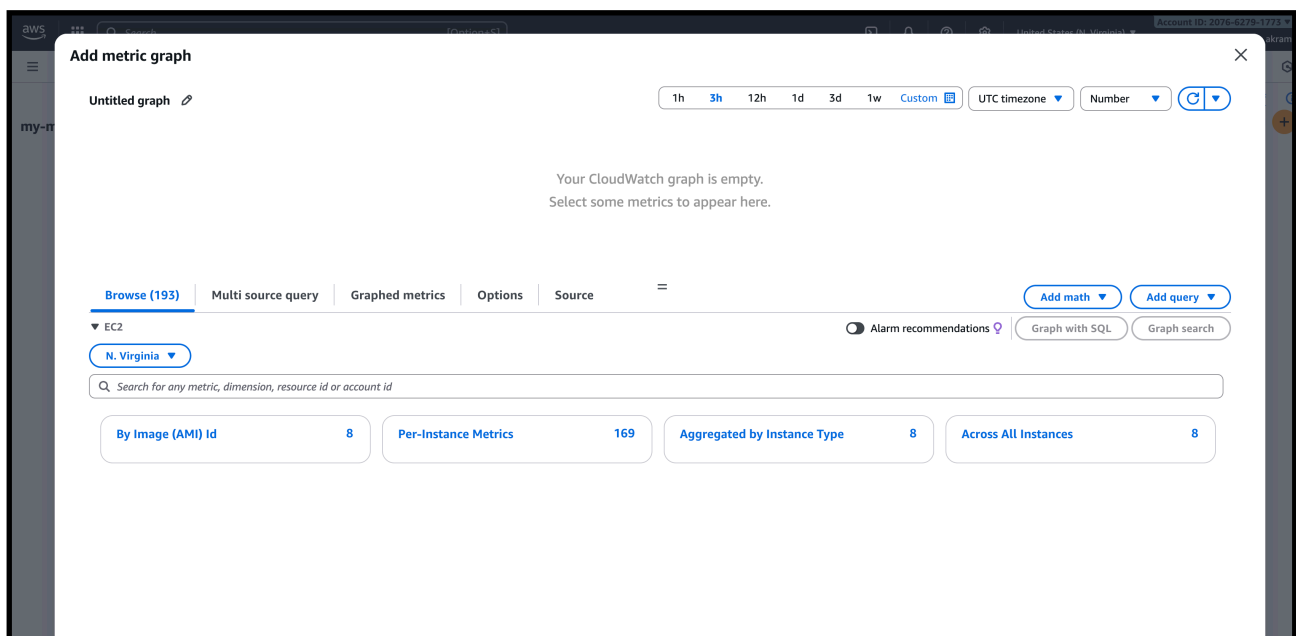
—> **Add Widget → Line Graph → CPU Metrics**

→ Click **Add widget**

→ Choose **number**

→ Choose **Metrics**

→ EC2 → Per-Instance Metrics



- Select BOTH instances:
- private-ec2 → CPUUtilization
 - public-ec2 → CPUUtilization

Untitled graph

1h3h12h1d3d1w

Your CloudWatch graph is empty.
Select some metrics to appear here.

Browse (8)

Multi source query

Graphed metrics

Options

Source

=

▼ Per-Instance Metrics

N. Virginia

Search for any metric, dimension, resource id or account id

CPUUtilization

Clear filters

☐	Instance name 8/8	▲ InstanceId	▼ Metric name	▼ Alarms
☐	No name specified	i-026e730dea2b1...	CPUUtilization ⓘ	No alarms
☐	No name specified	i-09b25d49c9e55...	CPUUtilization ⓘ	No alarms
☐	No name specified	i-0577c3409a642...	CPUUtilization ⓘ	No alarms
☐	No name specified	i-0170cc427834e6...	CPUUtilization ⓘ	No alarms
☐	No name specified	i-0df96800da9d8c...	CPUUtilization ⓘ	No alarms
☐	No name specified	i-0e63d5c43b038f...	CPUUtilization ⓘ	No alarms

—> select the both instances

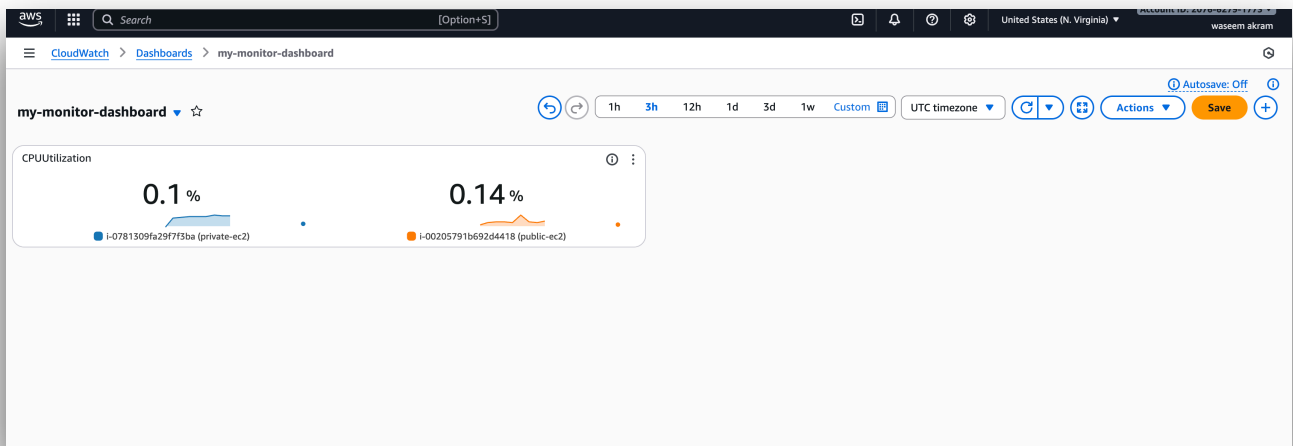
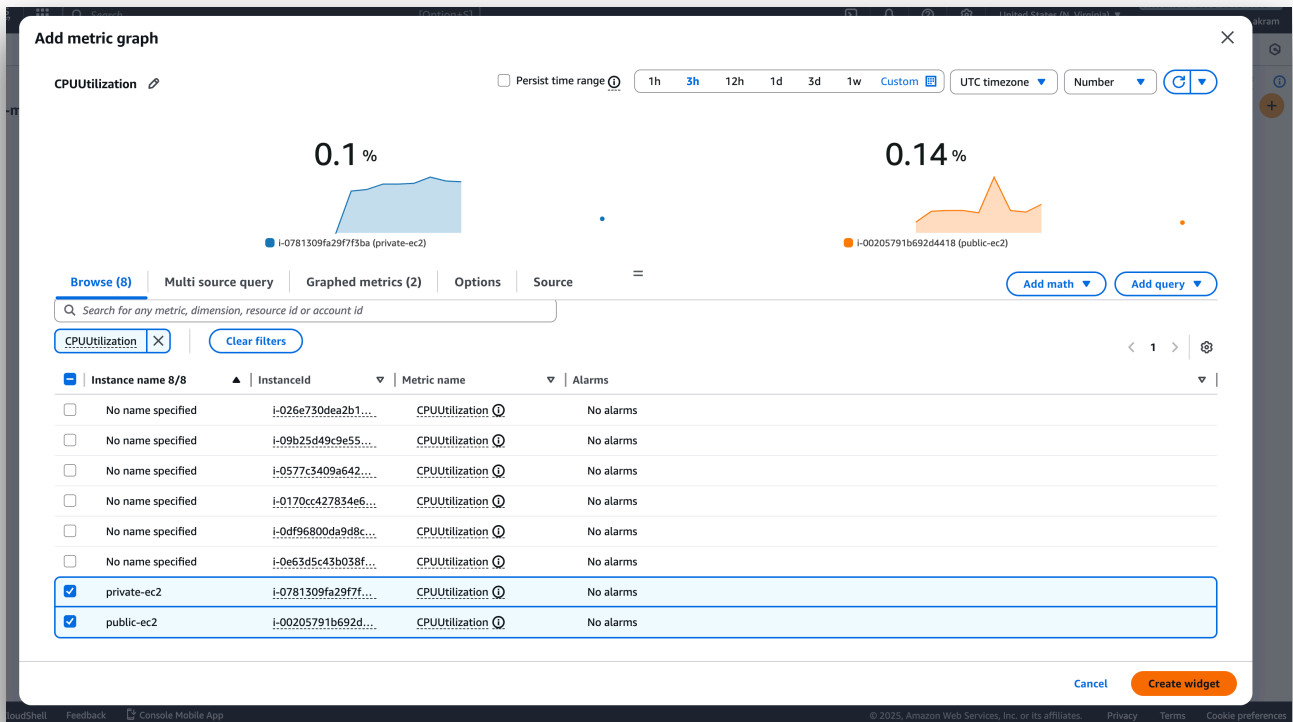
Search for any metric, dimension, resource id or account id

CPUUtilization

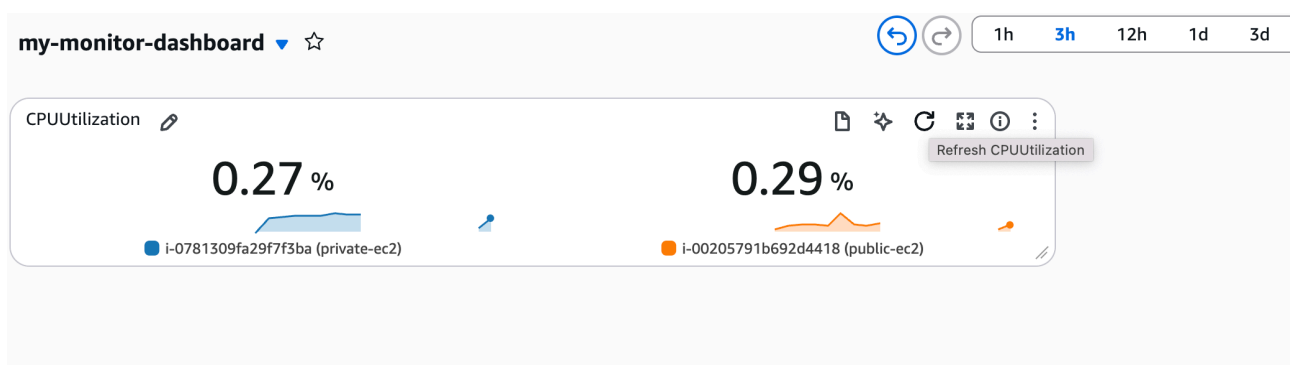
Clear filters

☒	Instance name 8/8	▲ InstanceId	▼ Metric name	▼ Alarms
☐	No name specified	i-026e730dea2b1...	CPUUtilization ⓘ	No alarms
☐	No name specified	i-09b25d49c9e55...	CPUUtilization ⓘ	No alarms
☐	No name specified	i-0577c3409a642...	CPUUtilization ⓘ	No alarms
☐	No name specified	i-0170cc427834e6...	CPUUtilization ⓘ	No alarms
☐	No name specified	i-0df96800da9d8c...	CPUUtilization ⓘ	No alarms
☐	No name specified	i-0e63d5c43b038f...	CPUUtilization ⓘ	No alarms
☒	private-ec2	i-0781309fa29f7f...	CPUUtilization ⓘ	No alarms
☒	public-ec2	i-00205791b692d...	CPUUtilization ⓘ	No alarms

—> click next



(A) CPU Utilization Monitoring for both EC2 Instances



We have created the dashboard for monitoring the cpuutilization of the two instances

B) —> Now lets create a dash board for the httpd monitoring

—> go inside the public ec2 instance and create a script to create a matric of the httpd

—> this below script will create a matric for httpd

—>. “opt/apache-monitor.sh” inside this add the script

```
#!/bin/bash

if systemctl is-active --quiet httpd; then
    STATUS=1
else
    STATUS=0
fi

aws cloudwatch put-metric-data \
  --namespace "CustomApache" \
  --metric-name "ApacheStatus" \
  --value $STATUS \
  --region us-east-1
```


Create cronjob to run every 1 minute

→ Then make it executable: `sudo chmod +x /opt/apache-monitor.sh`

```

#_
~\ ####_ Amazon Linux 2023
~~ \#####\
~~ \###|
~~ \#/ ____ https://aws.amazon.com/linux/amazon-linux-2023
~~ V~' '->
~~~~
~~~.-.-\
~~\ \
~/m/'

```

```

Package
=====
Installing:
cronie                                x86_64
Installing dependencies:
cronie-anacron                       x86_64

Transaction Summary
=====
Install 2 Packages

```

—> below is the list of our cronjob

```
complete.  
[ec2-user@ip-10-0-1-238 ~]$ sudo systemctl start crond  
sudo systemctl enable crond  
[ec2-user@ip-10-0-1-238 ~]$ sudo crontab -e  
no crontab for root - using an empty one  
crontab: installing new crontab  
[ec2-user@ip-10-0-1-238 ~]$ sudo crontab -l  
* * * * * /opt/apache-monitor.sh  
  
[ec2-user@ip-10-0-1-238 ~]$
```

—> after that we have to login to our aws account

—> with aws configure

—> and add all the access key details with region

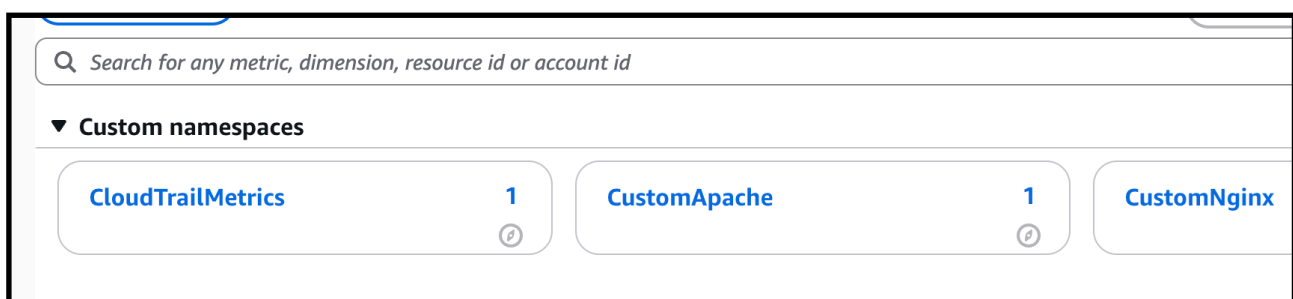
```
[AWS Secret Access Key [None]: eo/IVvxKm8ZSzh0GlByif/L55gK1ilUbZIfUI+q7  
eo/IVvxKm8ZSzh0GlByif/L55gK1ilUbZIfUI+q7  
[ec2-user@ip-10-0-1-238 ~]$ aws configure  
[AWS Access Key ID [None]: AKIATAWNKXROZALLH66G  
[AWS Secret Access Key [None]: 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  
[Default region name [None]: us-east-1  
[Default output format [None]: json  
[ec2-user@ip-10-0-1-238 ~]$ sudo vi /opt/apache-monitor.sh  
[ec2-user@ip-10-0-1-238 ~]$ /opt/apache-monitor.sh  
[ec2-user@ip-10-0-1-238 ~]$
```

—>And later run the script

/opt/apache-monitor.sh

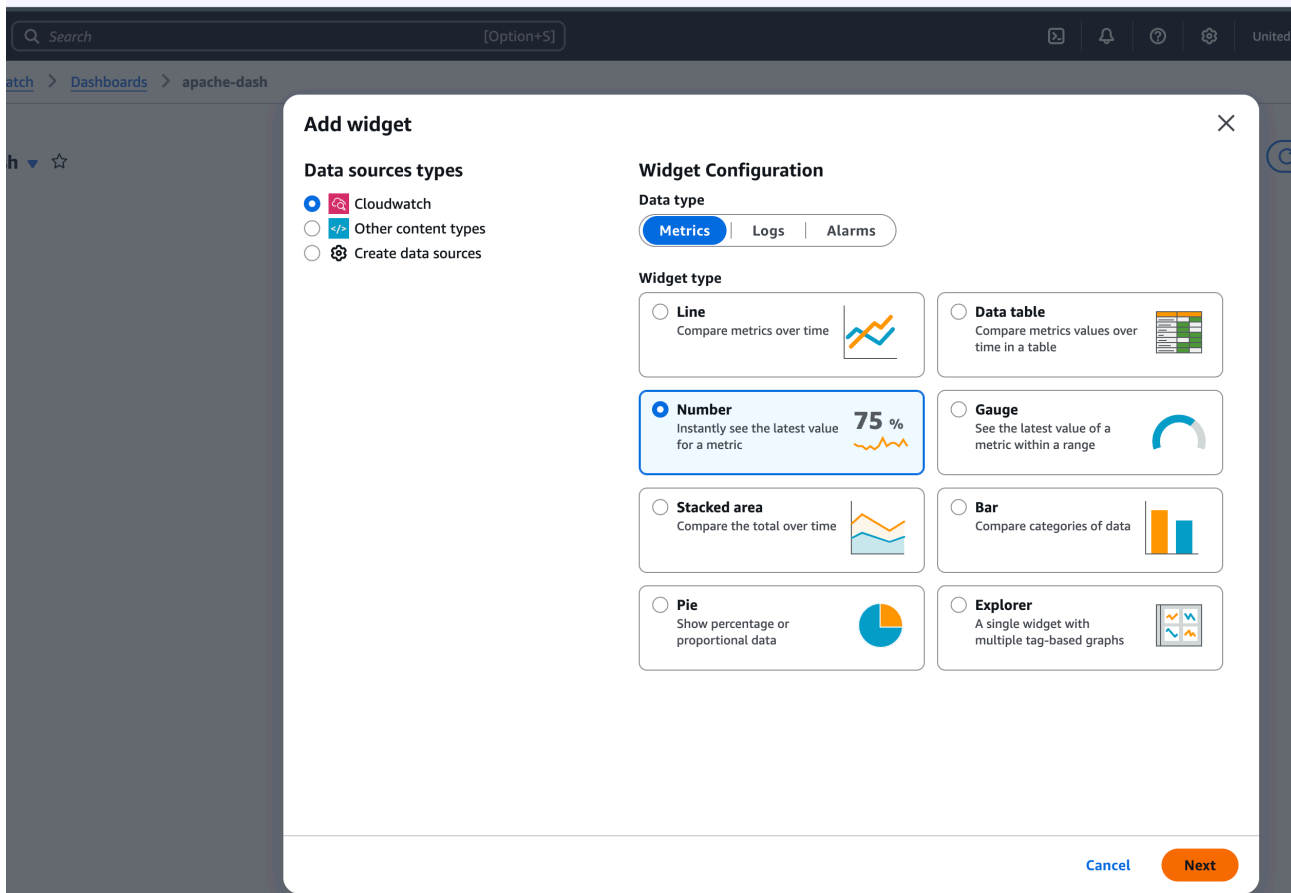
—> after running the script our metric for httpd (apache) will be created

—> to check go to cloudwatch—>all metric

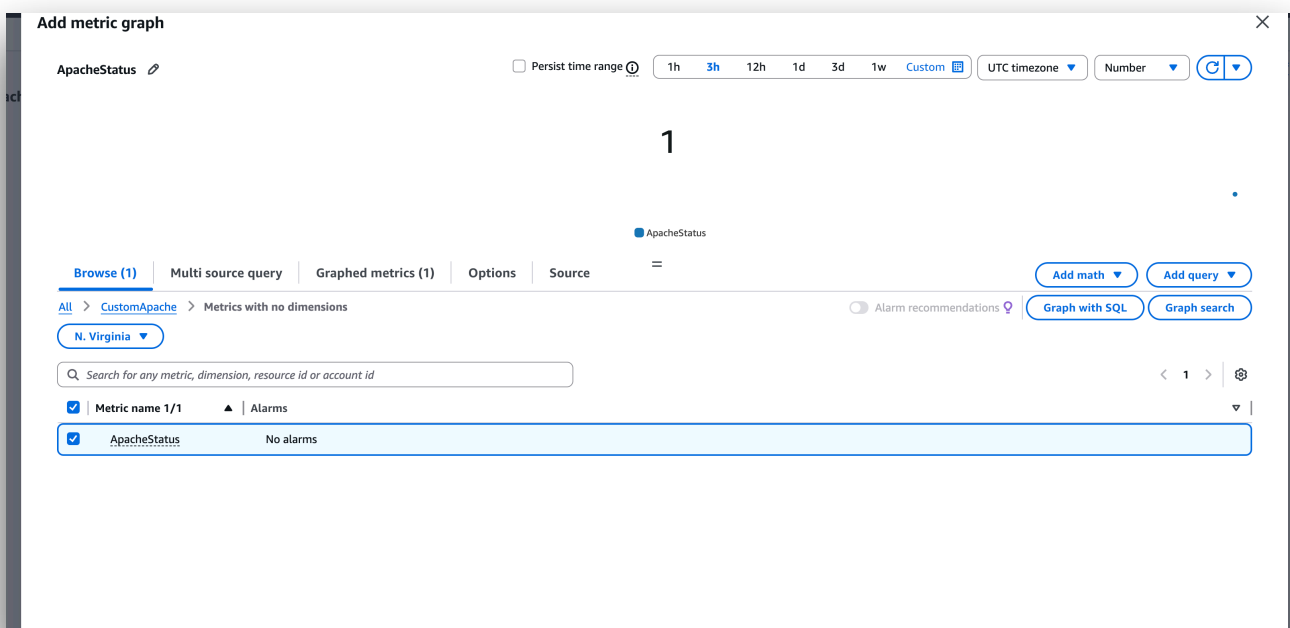


—-> now lets create a dash bord of the apache

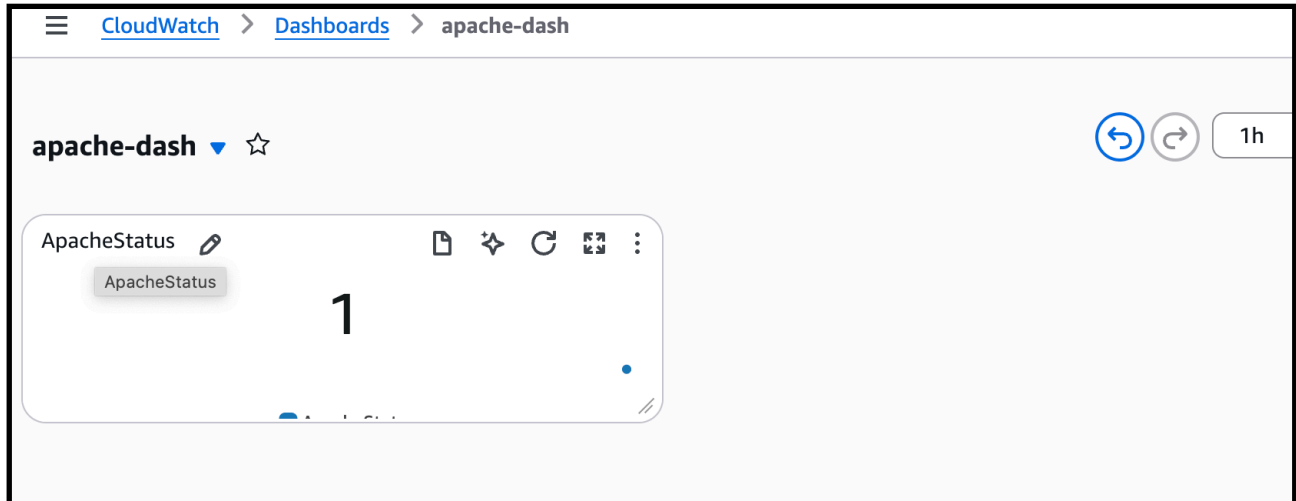
—-> dashbord —>create dashboard—>select the apache matrix



—> And create



—> we have now we can see the apache status as 1 because it is running



—> by this we have created dashboard for both
(A) CPU Utilization Monitoring for both EC2 Instances
(B) Apache HTTPD Service Monitoring

Custom Dashboards

Automatic dashboards

Custom Dashboards (2) Info

Q Filter dashboards

Share dashboard

Delete

Create dashboard

< 1 >

	Name	Sharing	Favorite	Last update (UTC)
☐	apache-dash		☆	2025-12-10 09:32
☐	my-monitor-dashboard		☆	2025-12-10 09:32

12) If CPU utilization is more than 70%, then it should trigger auto scaling and launch new instance.

To trigger auto scaling at 70% CPU:

We need 4 things:

- 1 **Launch Template** (your EC2 configuration)
- 2 **Auto Scaling Group (ASG)**
- 3 **Target Group already exists** (we used it for ALB) → we will attach ASG to this
- 4 **Scaling Policy: CPU > 70% → Add 1 instance**

—> we have to create a launch template

Create Launch Template

Go to:

EC2 → Launch Templates → Create launch template

Fill these:

≡ Search results

Create launch template

Creating a launch template allows you to create a saved instance configuration that can be reused, shared and launched at a later time. Template

Launch template name and description

Launch template name - *required*

Must be unique to this account. Max 128 chars. No spaces or special characters like '&', '*', '@'.

Template version description

Max 255 chars

Auto Scaling guidance | [Info](#)

Select this if you intend to use this template with EC2 Auto Scaling

☐ Provide guidance to help me set up a template that I can use with EC2 Auto Scaling

► **Template tags**

► **Source template**

Launch template contents

Specify the details of your launch template below. Leaving a field blank will result in the field not being included in the launch template.

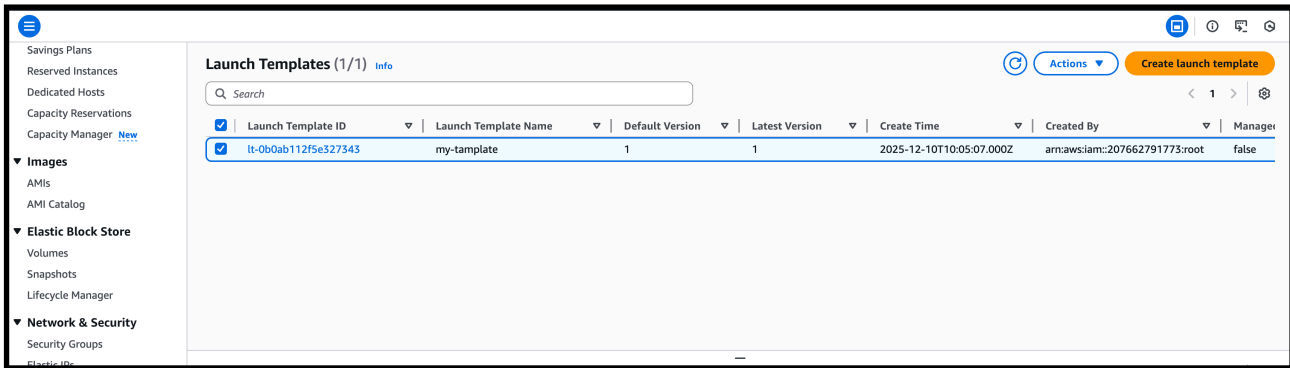
▼ **Application and OS Images (Amazon Machine Image)** [Info](#)

An AMI contains the operating system, application server, and applications for your instance. If you don't see a suitable AMI below, use the search bar to find one. [Browse more AMIs.](#)

—> **Template Name: my-template**

—> select the ami and key pair and SG group

—> and create the launch template

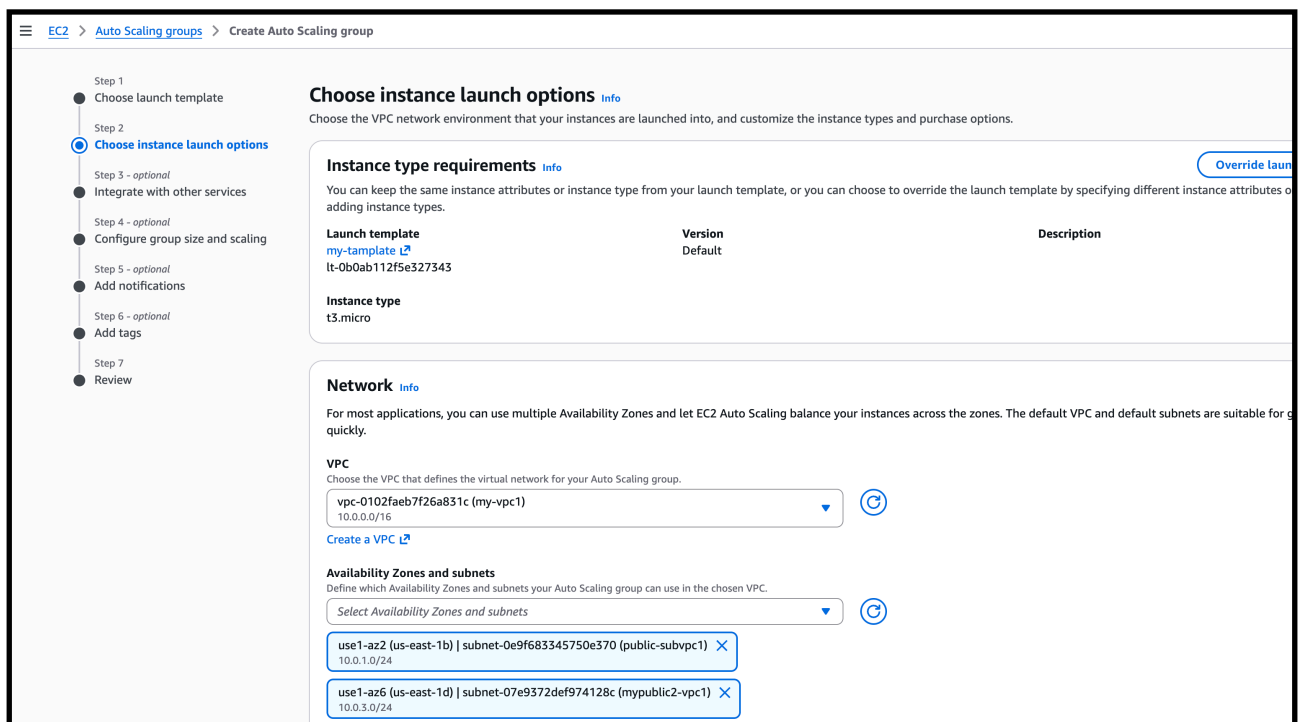


—> now we have to create a ASG (auto scaling group)

EC2 → Auto Scaling Groups → Create ASG

—> in very first select the launch template which we created

—> next select the vpc which we created and then select the availability zones of subnets from that vpc



—> click next

—> after that we have to
Load Balancing (IMPORTANT!)

Select:

✓ **Attach to an existing load balancer**

Choose the **target group** you created earlier (the one attached to ALB).

Example: my-target-group

—> This ensures new instances automatically register to ALB.

—>. **Set Group Size scaling**

Desired capacity: 2

Minimum capacity: 1

Maximum capacity: 4

The screenshot shows the 'Configure group size and scaling' step in the AWS Management Console. The left sidebar lists steps from 'Choose launch template' to 'Review', with 'Configure group size and scaling' selected. The main content area is titled 'Configure group size and scaling - optional' and includes sections for 'Group size', 'Desired capacity type', 'Desired capacity', 'Scaling', 'Scaling limits', and 'Automatic scaling'. The 'Desired capacity' is set to 2. The 'Scaling limits' show a minimum of 1 and a maximum of 4. The 'Automatic scaling' section has two options: 'No scaling policies' (selected) and 'Target tracking scaling policy'.

EC2 > Auto Scaling groups > Create Auto Scaling group

Step 1: Choose launch template
Step 2: Choose instance launch options
Step 3 - optional: Integrate with other services
Step 4 - optional: Configure group size and scaling
Step 5 - optional: Add notifications
Step 6 - optional: Add tags
Step 7: Review

Configure group size and scaling - optional [Info](#)

Define your group's desired capacity and scaling limits. You can optionally add automatic scaling to adjust the size of your group.

Group size [Info](#)
Set the initial size of the Auto Scaling group. After creating the group, you can change its size to meet demand, either manually or by using automatic scaling.

Desired capacity type
Choose the unit of measurement for the desired capacity value. vCPUs and Memory(GiB) are only supported for mixed instances groups configured with a set of instance attributes.

Units (number of instances) ▼

Desired capacity
Specify your group size.
2

Scaling [Info](#)
You can resize your Auto Scaling group manually or automatically to meet changes in demand.

Scaling limits
Set limits on how much your desired capacity can be increased or decreased.

Min desired capacity 1 Equal or less than desired capacity
Max desired capacity 4 Equal or greater than desired capacity

Automatic scaling - optional
Choose whether to use a target tracking policy [Info](#)
You can set up other metric-based scaling policies and scheduled scaling after creating your Auto Scaling group.

☒ **No scaling policies**
Your Auto Scaling group will remain at its initial size and will not dynamically resize to meet demand.

☐ **Target tracking scaling policy**
Choose a CloudWatch metric and target value and let the scaling policy adjust the desired capacity in proportion to the metric's value.

- > automatic scaling
- > select the target tracking scaling policy

Policy Settings

- **Metric type:** Average CPU Utilization
- **Target value:** 70

EC2 > Auto Scaling groups > Create Auto Scaling group

Specify your group size:

2

Scaling [Info](#)

You can resize your Auto Scaling group manually or automatically to meet changes in demand.

Scaling limits

Set limits on how much your desired capacity can be increased or decreased.

Min desired capacity **Max desired capacity**

1 4

Equal or less than desired capacity Equal or greater than desired capacity

Automatic scaling - optional

Choose whether to use a target tracking policy [Info](#)

You can set up other metric-based scaling policies and scheduled scaling after creating your Auto Scaling group.

☐ No scaling policies
Your Auto Scaling group will remain at its initial size and will not dynamically resize to meet demand.

☒ **Target tracking scaling policy**
Choose a CloudWatch metric and target value and let the scaling policy adjust the desired capacity in proportion to the metric's value.

Scaling policy name

Target Tracking Policy

Metric type [Info](#)

Monitored metric that determines if resource utilization is too low or high. If using EC2 metrics, consider enabling detailed monitoring for better scaling performance.

Average CPU utilization

Target value

70

Instance warmup [Info](#)

60 seconds

☐ Disable scale in to create only a scale-out policy

- > after next review the ASG and create the auto scaling group

EC2 > Auto Scaling groups > Create Auto Scaling group

Review [Info](#)

Step 1: Choose launch template [Edit](#)

Group details

Auto Scaling group name
launching-template

Launch template

Launch template	Version	Description
my-template l2	Default	

Step 2: Choose instance launch options [Edit](#)

Network

VPC
[vpc-0102faeb7f26a831c](#) [l2](#)

Availability Zones and subnets

Availability Zone	Subnet	Subnet CIDR range
use1-az2 (us-east-1b)	subnet-0e9f683345750e370 l2	10.0.1.0/24
use1-az6 (us-east-1d)	subnet-07e9372def974128c l2	10.0.3.0/24

Availability Zone distribution
Balanced best effort

- Watch CPU of all EC2 in the group
- If CPU > 70% for ~2 minutes
- It launches a new EC2 instance automatically
- The new EC2 attaches to:
 - ✓ Target Group
 - ✓ Load Balancer
 - ✓ VPC
 - ✓ Security Group

Everything works automatically.

—> check the instances based on cpu it will automatically launch the instance

☐	i-046cbf369e7cb2a3f	Running	t3.micro	Initializing	View alarms +	us-east-1b	-	-
☐	i-07edf3664281bdce3	Running	t3.micro	Initializing	View alarms +	us-east-1d	-	-
☐	public-ec2	Terminated	t3.micro	-	View alarms +	us-east-1b	-	-

—> so every thing is working automatically

Name	Instance ID	Instance state	Instance type	Status check	Alarm status	Availability Zone	Public
☐ private-ec2	i-0781309fa29f7f3ba	Running	t3.micro	3/3 checks passed	View alarms +	us-east-1b	-
☐	i-0d682bb75381f1a64	Terminated	t3.micro	-	View alarms +	us-east-1b	-
☐	i-0a5bc6f1d3714c547	Terminated	t3.micro	-	View alarms +	us-east-1b	-
☐	i-0f73469b33bec0503	Terminated	t3.micro	-	View alarms +	us-east-1b	-
☐	i-046cbf369e7cb2a3f	Terminated	t3.micro	-	View alarms +	us-east-1b	-
☐	i-02cd7dd8541fc5ae5	Terminated	t3.micro	-	View alarms +	us-east-1d	-
☐	i-07edf3664281bdce3	Terminated	t3.micro	-	View alarms +	us-east-1d	-
☐	i-0a19149ebf71787a0	Terminated	t3.micro	-	View alarms +	us-east-1d	-

Select an instance

